

Intrusion detection for Industrial Internet of Things based on deep learning

Yaoyao Lu, Senchun Chai*, Yuhan Suo, Fenxi Yao, Chen Zhang

School of Automation, Beijing Institute of Technology, Beijing 100081, China

ARTICLE INFO

Communicated by R. Razavi-Far

Keywords:

Intrusion detection
IIoT
Data imbalance
Feature selection
Deep learning

ABSTRACT

Intrusion detection technology can actively detect abnormal behaviors in the network and is important to the security of Industrial Internet of Things (IIOT). However, there are some issues with the current intrusion detection technology for IIOT, such as extreme imbalance in the number of samples of different classes in the dataset, redundant and meaningless features in the samples, and the inability of traditional intrusion detection methods to meet the requirements of detection accuracy in the increasingly complex IIOT. In view of the extreme imbalance of classes, this paper applies the hierarchical clustering algorithm to the under-sampling technology, which reduces the number of majority samples while reducing the loss of information of majority samples, and solves the problem of missing detection and false detection of minority samples caused by sample imbalance. In order to avoid feature redundancy and interference, this paper proposes an optimal feature selection algorithm based on greedy thought. This algorithm can obtain the optimal feature subset of each type of data in the data set, thus eliminating redundant and interfering features. Aiming at the problem of insufficient detection ability of traditional detection methods, this paper proposes a deep neural network intrusion detection model based on the parallel connection of global and local subnetworks. This model obtains the overall benchmark detection of the dataset through the deep neural network, and then strengthens the detection effect of each subclass through the parallel connection of subnetworks, greatly improving the performance of the intrusion detection algorithm. The experimental results show that the method described in this paper can improve the intrusion detection for IIOT.

1. Introduction

With the expansion of Internet technology application scenarios and the transformation of traditional industrial manufacturing systems, as well as the development of 5G (fifth generation) communication technology, IIOT is moving towards a larger scale and deeper network interconnection, and plays an increasingly important role in the new generation of industrial reform [1–6]. In this situation, IIOT will become a bridge connecting the traditional industrial system and the new Internet industry, providing basic support for the upgrading of traditional industries.

IIOT is mainly composed of physical service systems and some digital devices such as sensors and actuators. It is mainly used to manage all kinds of data generated in the process of industrial production and realize the intelligence of industrial production and management. During the COVID-19 pandemic, against the background of isolation and economic turmoil, the development of IIOT has played a vital role in ensuring the efficiency of production management and the continuity of production.

However, on the one hand, due to the large amount of data stored in IIoT, the risk of IIoT being attacked by cyber attacks is greatly increased, which poses a great threat to the management and production

of enterprises. On the other hand, with the continuous development of IIOT, more and more industries have joined it, making its network structure more and more complex, so the requirements for its defense capabilities are also increasing [7–10]. Under the combined effect of these two factors, the security issue of IIOT has become more urgent, and the research on intrusion detection technology applied to IIOT has also become a current hot spot.

Intrusion detection is a proactive security protection technology that can automatically detect and report any abnormal traffic that may affect IIoT network security, including internal attacks, external attacks, and misoperations [11]. When abnormal traffic occurs in IIOT, the use of intrusion detection technology can quickly and accurately determine the type of abnormal traffic and take corresponding defense measures to block it before the network system is violated. Therefore, intrusion detection plays a key role in protecting IIoT network security, ensuring data integrity and reliability, and ensuring normal industrial production [12]. There are two key issues to be solved in the current intrusion detection technology applied to IIOT, that is, how to extract network behavior and how to distinguish network behavior.

On the issue of how to extract network behaviors, some researchers introduced network traffic analysis technologies such as deep packet

* Corresponding author.

E-mail address: chaisc97@bit.edu.cn (S. Chai).

analysis technology to capture the original traffic from each network node and parse the content of different levels of messages to achieve traffic detection, for example, Markel, etc. applied deep packet analysis technology to successfully detect ICMP packet attacks in software-defined networks [13]. Quamar et al. achieved DDoS attack detection by filtering the header information of the original traffic packet [14]. Some researchers proposed statistical methods to process network traffic information and employed it for subsequent analysis, for example, Moustafa et al. in the process of creating the UNSW NB15 data set, performed secondary processing on the traffic according to basic attributes, flow attributes, content attributes, etc., and employed this as the subsequent classification data set [15].

However, the current public intrusion detection datasets used as benchmark data in the research field generally have extremely unbalanced numbers of samples of different classes, and the actual network traffic in IIOT also shows this characteristic. At the same time, in the current complex IIOT network environment, in order to more comprehensively evaluate the network status, it is usually necessary to collect the traffic of multiple nodes, which leads to a high dimensionality of data features. But when it comes to the description of a certain type of network behavior, most of the features are redundant or even interfering. Therefore, the above methods are not effective in detecting abnormal traffic in IIOT.

In order to alleviate the problem of class imbalance, some scholars have carried out research in this area. Suo et al. [16] integrated the K-means algorithm on the basis of the traditional SMOTE algorithm, and used a hybrid oversampling and undersampling method to alleviate the class imbalance problem. Liu et al. [17] combined ensemble of classifier chains with random undersampling methods to propose a method to alleviate class imbalance in datasets. Agustianto et al. [18] introduced the Neighborhood Cleanup Rule based on the Decision Tree algorithm, and proposed an NCL-based sampling method to deal with unbalanced data in the dataset. Zanjani et al. [19] proposed a new generative adversarial framework for learning imbalanced class distributions. Gupta et al. [20] proposed LIO-IDS based on long short-term memory (LSTM) classifier and improved one-to-one technique to deal with unbalanced network intrusions.

In the face of a large number of redundant and meaningless features in data sets, Al Dawei et al. applied rough set theory (RST) to comprehensively analyze the features of KDD99 and UNSW-NB15 data sets, calculated the dependency ratio between features and classes, and extracted the features of the data set on this basis [21]. Samat et al. [22] proposed a feature selection algorithm based on the Minimum Redundancy Maximum Relevance algorithm, which reduced the interference and redundant features in the data set. Tschannen et al. [23] used Mutual Information Maximization (MIM) to build a feature extractor, and trained the feature extractor by maximizing an estimate of the mutual information (MI) between different views of the data, and finally improved the data sets and achieved good results. In order to improve the feature space for data sets, Hallaji et al. [24] proposed a dimensionality reduction algorithm based on generative adversarial networks, which reduced redundant and meaningless features in data sets by transforming the feature space into a low-dimensional space. The authors of [25] introduced dimensionality reduction technology in Intrusion Detection System (IDS), which enhanced the detection performance of IDS. Some researchers used deep learning methods to directly extract the features of the data. For example, Zong et al. [26] reconstructed the original data using a depth self-coder to extract the higher-order coding expression of the data in the middle layer, thus improving the overall detection accuracy.

In addition, the effect of intrusion detection will also be affected by some other factors. For example, the authors of [27] proposed a defense mechanism based on adversarial training and label noise analysis to make IDS robust against bad training data in IoT and enhanced the detection performance of IDS. Ref. [28] proposed a stream learning approach for real-time identification of false data injection attacks to

tackle the issue of concept drift in IDS, thus helping IDS to better identify network attacks in the system. However, as the network structure of IIOT becomes more and more complicated, its complexity and data volume continue to increase, how to better alleviate the class imbalance in the dataset and extract data features is still a challenging problem.

As for how to distinguish network behavior, its essence is the application of classification problems in the field of intrusion detection. For the classification stage only, some researchers applied statistical methods to calculate a set of discriminative quantities, and then distinguished the types of network behaviors through the discriminative quantities. For example, Qu et al. combined the Self Organizing Mapping algorithm and the Inner Distribution Factor statistical parameters and applied the statistical parameters to distinguish the types of samples in the incompletely split neurons, thereby improving the detection accuracy [29]. Some researchers introduced machine learning and deep learning classifiers to realize the distinction [30–37], for example, Wu et al. proposed convolutional neural networks, Yin et al. proposed recurrent neural networks, and Lan et al. employed a two-way LSTM network to implement the classification process of intrusion detection. However, the above works did not apply the deep learning algorithm to the intrusion detection for IIOT in a targeted manner, and cannot meet the requirements of detection accuracy.

To sum up, there are some issues with the current intrusion detection technology for IIOT, such as extreme imbalance in the number of samples of different classes in the dataset, redundant and meaningless features in the samples, and the inability of traditional intrusion detection methods to meet the requirements of detection accuracy in the increasingly complex IIOT. In order to solve the above problems, this paper conducts the following research:

- In view of the extreme imbalance of the classes in the data set, this paper first proposes a nonlinear logarithmic transformation method to improve the traditional normalization method to better handle the high dispersion characteristics of the data set. Then, this paper applies the hierarchical clustering algorithm to the under-sampling algorithm, makes some improvements to the selection of the target cluster number, the design of the sampling strategy, and the clustering criteria, and puts forward an under-sampling algorithm based on hierarchical clustering. Finally, this paper also proposes a data balancing algorithm framework, to combine with other sampling algorithms to obtain a more reasonable data set with completely balanced classes.
- In view of the redundant and meaningless features in the data set, this paper first introduces the entity embedding mapping of categorical features to solve the problems of the traditional one-hot mapping method, such as too large feature dimension and sparse data space after mapping. Then, this paper proposes a new optimal feature selection algorithm combining greedy thought and cross-validation algorithm, which can obtain the optimal feature subset of a certain type of data, thereby eliminating redundant and meaningless features.
- Aiming at the problem of insufficient detection ability of traditional classifiers, this paper introduces the Principal Component Analysis method and then combined with deep learning, proposes a deep neural network intrusion detection model based on the global and local subnetwork parallel based on the data balance algorithm and feature selection algorithm previously proposed. Finally, we design relevant simulation experiments to verify the advantages of the proposed algorithms and models.

2. Research program

2.1. Data balancing

2.1.1. Data preprocessing

Before conducting the research in this article, it is necessary to normalize the high-dimensional data set. The normalization methods

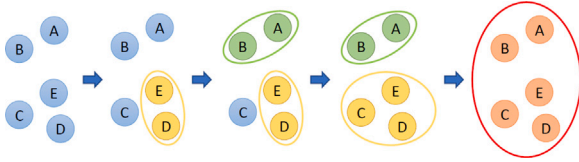


Fig. 1. The hierarchical clustering algorithm iterative process schematic diagram.

commonly used in the existing research include MinMax normalization, Z-score normalization, Robust normalization, or a combination of the above methods [38], which are linear. However, it is not appropriate to adopt the above standardization method uniformly for all data sets.

When some characteristic data in the data set obey a seriously skewed distribution or the deviation of the characteristic data is particularly large, if the linear normalization operation is carried out for such features, the non-zero value near the zero point of the feature will be compressed into a very small value. This means that most of the values of the feature will be in a minimum neighborhood of the zero point after the linear normalization process, which leads to the lower discrimination of the feature compared with other features. It can be known that after linear normalization, although the high dispersion feature is adjusted to the same level as the low dispersion feature, the information content of the former is substantially reduced. Therefore, the MinMax normalization method is improved in this paper. First, the logarithmic transformation is performed on the high dispersion features in the data set, and then the MinMax normalization operation is performed on all features [39]. The formula is as follows:

$$X_{iLogMinMax} = \begin{cases} \frac{\log(1+X_i) - \log(1+X_{imin})}{\log(1+X_{imax}) - \log(1+X_{imin})} & \sigma(X_i) \geq 1, \\ \frac{X_i - X_{imin}}{X_{imax} - X_{imin}} & \sigma(X_i) < 1, \end{cases} \quad (1)$$

where, X_i is the vector composed of all the values of the i th feature in the data set, X_{imax} is the maximum value in X_i , X_{imin} is the minimum value in X_i , and $\sigma(X_i)$ is the standard deviation of X_i . In this paper, it is specified that the characteristics that meet the requirements of $\sigma(X_i) \geq 1$ are high dispersion characteristics.

2.1.2. The hierarchical clustering algorithm based on VSS gain

There are two ways to solve the problem of sample imbalance at the data level, including oversampling the minority class samples and undersampling the majority class samples. The former is easy to cause over-fitting, and for extremely unbalanced data sets, the use of this method often fails to achieve the desired effect; the latter will cause a considerable part of data feature information to be lost. The main research in this section is to undersample the majority class samples, so in order to solve the problem of feature information loss caused by the undersampling method, this paper introduces the hierarchical clustering algorithm and proposes a hierarchical clustering algorithm based on VSS Gain on the basis of the original algorithm.

The hierarchical clustering algorithm first assumes that each sample of the dataset is an independent cluster. In each iteration of the algorithm, the two clusters closest to each other are selected for merging until the entire dataset is merged into one cluster, that is, the entire dataset itself. This process is shown in Fig. 1.

Therefore, for the hierarchical clustering algorithm, the most important step is to calculate the distance between two clusters during each iteration, find and merge the two clusters with the closest distance [40]. This paper gives a calculation equation $d_{cluster}$ for cluster spacing based on the gain of variance sum of samples (VSS Gain), the derivation process is as follows.

Given a dataset cluster C , which contains m n -dimensional column vectors X_i composed of all the values of the i th feature in the data set, C and X_i are shown below:

$$C = \{X_1, X_2, \dots, X_m\}, \quad (2)$$

$$X_i = \{x_{i1}, x_{i2}, \dots, x_{in}\}^T, \quad (3)$$

where, $i \in [1, \dots, m]$, $j \in [1, \dots, n]$ m represents the number of features in the data set, n represents the number of samples in the data set, and x_{ij} represents the value corresponding to the i th feature of the j th sample in the data set.

The concept of variance sum of samples in cluster (VSS) is defined as follows:

$$VSS = \sum_{i=1}^m d(X_i, X_c)^2, \quad (4)$$

where, X_c is the center point of the cluster mean value, $d(X_i, X_c)$ is the Euclidean distance between vector X_i and X_c :

$$d(X_i, X_c) = \sqrt{(x_{i1} - x_{c1})^2 + \dots + (x_{in} - x_{cn})^2} \quad (5)$$

$$X_c = [x_{c1}, x_{c2}, \dots, x_{cn}]^T \quad (6)$$

$$x_{cj} = \frac{\sum_{i=1}^m x_{ij}}{m}, j \in [1, 2, \dots, n]$$

Assuming that there are currently two clusters C_1 and C_2 to be merged, the total variance sum of samples of the new clusters after merging is:

$$VSS_{C_1 \cup C_2} = \sum_{X \in C_1 \cup C_2} d(X, X_c \in C_1 \cup C_2)^2. \quad (7)$$

Based on the above, this paper defines the distance between two clusters $d_{cluster}$ as the gain of variance sum of samples before and after their combination, as follows:

$$d_{cluster}(C_1, C_2) = VSS_{C_1 \cup C_2} - VSS(C_1) - VSS(C_2). \quad (8)$$

At this point, the goal of the hierarchical clustering algorithm is to merge the two clusters with the smallest cluster spacing among all the current clusters, that is, to merge the two clusters with the smallest gain of variance sum of samples. The process of VSS Gain-based Agglomerative Hierarchical Clustering Algorithm (VG-AGHC) is shown in Algorithm 1.

Algorithm 1 The Hierarchical Clustering Algorithm Based on VSS Gain (VG-AGHC)

Input: Dataset $S = S_1, S_2, \dots, S_m$.

Output: Hierarchical clustering tree $Tree(S)$.

The number of clustering clusters is defined as *group*, and the initial value is the number of samples, that is, each sample is regarded as a cluster.

for $group = m \rightarrow 1$ **do**

At this time, the Dataset $S = C_1, C_2, \dots, C_{group}$, where C_i is the i -th cluster.

for $i = 1 \rightarrow group$ **do**

Calculate the $VSS(C_i)$ according to Equation (4).

end for

Calculate the distance between each cluster by Equation (8).

Construct the distance critical table $T_{cluster}$.

$T_{cluster}(i, j) = d_{cluster}(C_i, C_j)$, and the diagonal elements are set to infinity.

Calculate $\min(T_{cluster}(i, j))$, and merge the corresponding C_i cluster and C_j cluster.

end for

All samples are leaf nodes, S is the root node, and the merging process is the branch of $Tree(S)$.

Return $Tree(S)$.

The data set can be built into a hierarchical clustering tree through the above algorithm. The bottom leaf node of the clustering tree is all sample individuals in the data set, and the top root node is the entire data set. From the leaf node to the root node, the number of nodes in each layer is the number of clusters corresponding to the current layer.

After getting the clustering tree of the dataset, we only need to determine the number of target clusters to complete the hierarchical clustering of the dataset. This paper proposes a method of using the weighted average standard deviation to obtain the target number of clusters.

According to the theoretical derivation in the previous section, the hierarchical clustering algorithm is a bottom-up greedy algorithm. The goal of each clustering merging is to minimize the VSS Gain after merging, in other words, make the merging clusters have the smallest variance sum of samples among all the current selections. At the beginning of the clustering algorithm, each sample is an independent cluster, and the sample variance sum is 0; in the middle stage of the clustering process, each aggregation selects the direction that minimizes the VSS Gain, that is, the direction that reduces the variance sum of samples; finally, it is combined into a complete dataset, and the variance sum of samples is a value not lower than the intermediate stage. According to the description of the above clustering process, it can be considered that the overall change trend of the variance sum of samples in the clustering process is increasing first and then decreasing. Therefore, from the perspective of the end point of hierarchical clustering, the inflection point of the descent curve of the variance sum of samples represents the number of target clusters.

In order to more accurately reflect the change of information content in the clustering process, this paper defines the weighted average standard deviation ($waStd$) to find this inflection point. Assuming that there are k clusters in the clustering data set S at this time, then:

$$\omega_i = \frac{\text{num}(C_i)}{\sum_{j=1}^k \text{num}(C_j)}, \quad (9)$$

$$waStd(S) = \sum_{i=1}^k \omega_i * \sigma(C_i). \quad (10)$$

Where, $i, j \in [1, \dots, k]$, C_i is the i th cluster in S , $\sigma(C_i)$ is the standard deviation of C_i , $\text{num}(C_i)$ represents the number of sample points in the cluster C_i , and ω_i is the ratio of the number of sample points in the cluster C_i to the number of all sample points in the data set. At this time, we only need to find the inflection point of the descent curve of the function $waStd$ to get the number of target clusters.

2.1.3. Majority under-sampling algorithm based on VG-AGHC

After the weighted average standard deviation method is used to determine the number of clusters (the target number of clusters) and the hierarchical clustering algorithm is used to obtain the corresponding number of sub-classes (clusters), the specified number of samples should also be extracted from these sub-classes.

This paper proposes three sampling strategy schemes.

- (1) Truncated sampling strategy: This strategy focuses on maintaining the balance of sub-samples. In each subclass, the same number of samples are extracted. After under-sampling, the interior of the set of samples of majority classes is balanced.
- (2) Boundary preservation sampling strategy: This strategy focuses on preserving the boundary information. Keep the boundary points of all subclassed (whether it is a boundary point can be determined by the K neighbor category), and then randomly sample the other majority class points, and the boundary of each subclass is maximized.
- (3) Equal proportion sampling strategy: This strategy focuses on keeping the proportions of each subclass constant. According to the number of subclasses after clustering, samples are extracted in equal proportions. After under-sampling, the sample distribution and proportions within the majority class will not be destroyed.

The above three sampling strategies are shown in Fig. 2.

Relying on the previous content, this paper proposes the Under-sampling Algorithm Based on VG-AGHC (VAGCUS). Using this algorithm to perform an undersampling operation on the majority class

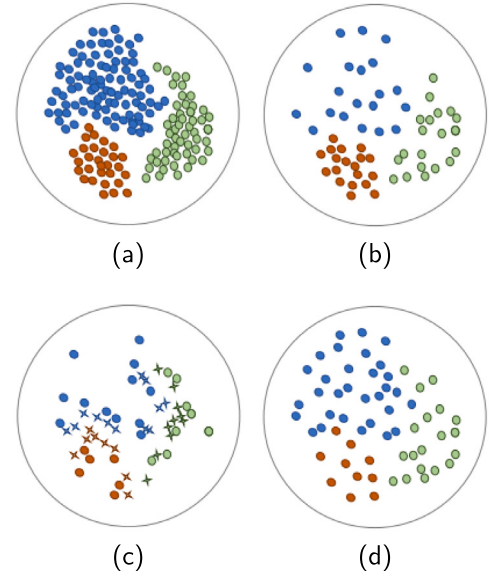


Fig. 2. Sampling strategy diagram: (a) Raw unsampled cluster data; (b) Truncated sampling strategy; (c) Boundary-preserving sampling strategy; (d) Proportional sampling strategy.

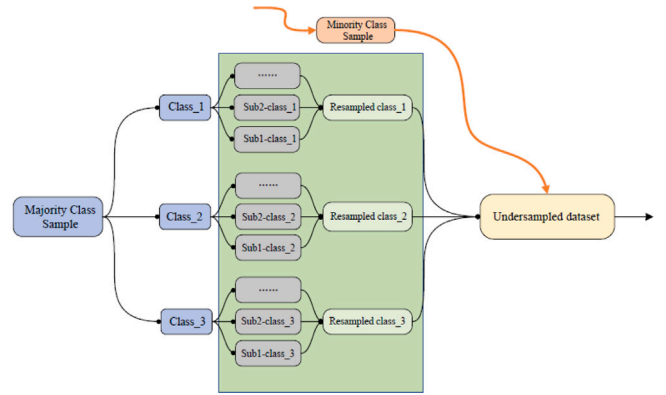


Fig. 3. Schematic diagram of the process of performing hierarchical clustering undersampling on data sets.

samples in the dataset can reduce the number of samples while preserving the main information of the majority class samples, so as to effectively alleviate the problem of missing detection of a few classes caused by the extreme imbalance of the dataset. The process of VAGCUS is shown in Algorithm 2.

The introduction of the hierarchical clustering algorithm can find the sub-class hidden in the majority class samples in the dataset. Then, when performing the undersampling operation, each majority class sample set is regarded as an independent individual, and the undersampling samples are obtained from its hidden sub-classes. That is, each cluster obtained by using hierarchical clustering in this section can be considered as a subclass of the majority class sample sets. In this way, the information and diversity of data can be preserved as much as possible while reducing the number of samples in majority class sample sets. For a general dataset, the process of implementing the hierarchical clustering undersampling algorithm for its majority class samples is shown in Fig. 3.

2.1.4. Data balancing algorithm framework based on VAGCUS

In practical applications, to obtain a more reasonable class-completely balanced data set, this paper combines the VAGCUS

Algorithm 2 Majority Under-sampling Algorithm Based on VG-AGHC (VAGCUS)

Input: The normalized data set $D = D_1, D_2, \dots, D_k$ containing only the majority class samples, the number of sampling target samples is n , sampling strategy: *strategy* = 1 -truncated type, 2-boundary preservation type, 3-equal ratio type.

Output: The majority class balanced data set after undersampling is $D_{resampled}$ and the number of samples in each type is n .

- 1: Define k as the number of classes, and $D_t = X_{t1}, X_{t2}, \dots, X_{tm}$ as the t -th majority class set.
- 2: Define m as the total number samples in the data set, and m_t as the total number of samples in the t -th majority class.
- 3: **for** $t = 1, 2, \dots, k$ **do**
- 4: Apply VG-AGHC algorithm to generate $Tree(D_t)$,
- 5: Define l_t as the number of clusters in D_t .
- 6: **for** $l_t = 1, 2, \dots, m_t$ **do**
- 7: Calculate $waSted(D_t)$ according to formula (10).
- 8: Jump out of the loop at the inflection point. At this time, l_t is the number of clusters in this class.
- 9: **end for**
- 10: Define the number of samples of the j -th cluster before and after sampling as s_{ij} and r_{ij} , respectively.
- 11: **if** *strategy* = 1 -truncated type **then**
- 12: Let the number of samples per cluster r_{ij} be equal to n/l_t .
- 13: All clusters whose number of samples are less than this value are sample and filled evenly from other clusters.
- 14: **end if**
- 15: **if** *strategy* = 2 -boundary preservation type **then**
- 16: Apply the KNN algorithm to calculate the boundary points in D_t , where the of k is 3.
- 17: Keep all the boundary points and calculate the number of remaining samples $n' = n - n_{borderline}$
- 18: Randomly sample n' samples from all clusters.
- 19: **end if**
- 20: **if** *strategy* = 3 -equal ratio type **then** Calculate the sampling ratio $factor = m_t/n$
- 21: **for** $j = 1, 2, \dots, l_t$ **do**
- 22: $r_{ij} = s_{ij} \cdot factor$.
- 23: Randomly sample samples from the j -th cluster.
- 24: **end for**
- 25: Obtain the sampling subset of the t -th majority class.
- 26: **end if**
- 27: **end for**
- 28: Combine the under-sampling subsets of the above-mentioned majority classes to obtain the overall under-sampling data set $D_{resampled}$.
- 29: **Return** $D_{resampled}$.

algorithm with the data generation algorithm, and proposes a data balancing framework based on the VAGCUS algorithm. The core of the framework is still to apply the aforementioned VAGCUS to perform under-sampling operations on the majority class samples in the extremely unbalanced data set. At the same time, as a supplement, the data generation algorithm is employed to generate the number of minority samples to be the same as the number of majority samples after sampling. The process of using the data balancing algorithm framework to obtain a fully balanced dataset is shown in Fig. 4.

2.2. Feature selection

2.2.1. Entity embedding of features

At present, most of the data sets related to network security contain non-numeric and non-size categorical features, such as flag, service, and protocol features contained in NSL-KDD. The values of these features

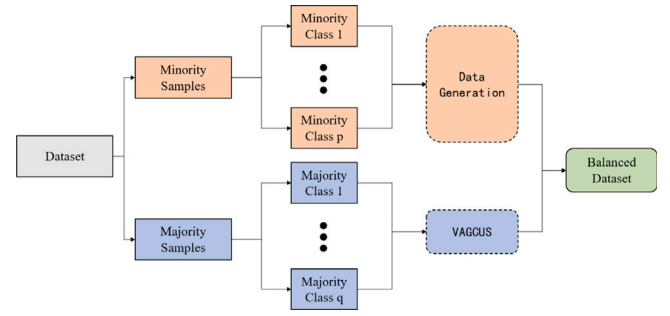


Fig. 4. Schematic diagram of executing data balancing algorithm framework.

are the names of corresponding protocols, flags, or services. Although there are differences in frequency, these nominal values themselves have no direct numerical relationship. These features contain a large amount of redundant information. If they are directly used without processing, the expected detection effect cannot be achieved.

A common method is to use One Hot Encoding to map the categorical features. One Hot Encoding can map the categorical features containing N values to N -dimensional independent One-hot vectors, the mapping dimension corresponding to the value is set to 1, and the other mapping dimensions are set to 0, which solves the problem of mathematical expression of non-numerical and non-magnitude relational features. Moreover, One Hot Encoding does not require prior knowledge, and the feature values are independent of each other and the mapping is unique.

However, all the values of the categorical feature will lose the relative relationship between the categorical features after being mapped into the independent vector by One-hot, which is reflected in the data space as the loss of the distance and position relationship between the feature values. Moreover, One Hot Encoding will create a feature dimension for each value of the categorical feature, which will lead to a significant increase in the processed feature dimension and sparse data space. If the model is trained with the method of deep learning, this coding method will lead to a large number of parameters to be trained, which will cause two problems to the model: the increase of parameters will increase the demand for data volume of the model; The increase of parameters leads to the increase of calculation of model training.

To solve the above problems, this paper introduces Entity Embedding [41]. Embedding is a method of mapping high-dimensional sparse data to low-dimensional dense data. The process is as follows:

$$f(\cdot) : x_{ij} \rightarrow E_{ij}, \quad (11)$$

where x_{ij} represents the i th value of the categorical feature, $f(\cdot)$ represents the embedding mapping process, and E_i represents the embedding vector after mapping.

This mapping can be understood as linearly mapping the categorical features after Onehot encoding, which is equivalent to adding a layer of linear fully connected neural network on the basis of Onehot, as shown below:

$$E_i = \sum_{j=1}^n \omega_{ij} \delta_{ij} = \omega_{x_i}, \quad (12)$$

among them, ω is the weight of the fully connected network, i is the serial number of the categorical feature value, j is the dimension serial number of the embedding vector, and δ_{ij} is the Kronecker delta function, which is the mathematical expression of One-hot.

Then, we can train a deep neural network to obtain the embedding matrix of the categorical features, and obtain the embedded representation of the categorical features. This section builds a multi-layer fully connected network with embedded layers, as shown in Fig. 5.

The embedded weights of the network during training convergence are a set of appropriate embedded representations. In the figure, the

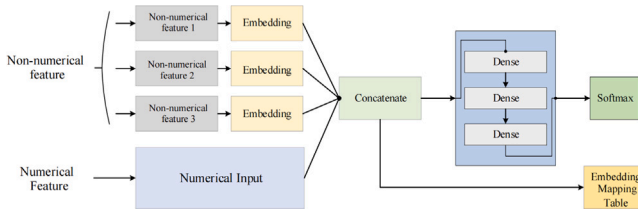


Fig. 5. Structure diagram of neural network including Embedding layer.

Concatenate layer splices the Embedding layer and the original input layer into complete input data, and the number in brackets represents the input dimension; Dense represents the full connectivity layer, and the number in brackets represents the number of neurons. The features of the data set after Embedding mapping can well retain the distance and position relationship between the features in the original data set, and also reduce the feature dimension compared with using only One-hot, which is very helpful to improve the detection effect.

2.2.2. Optimal feature selection algorithm

All intrusion detection data sets contain features of different meanings and categories. The most basic features are the direct features in the traffic package, such as protocol type, service type, duration, etc., and some higher-order features are obtained by combining log information. In general, both the public network security data set and the self-made traffic data set will retain as many features as possible in the collection stage, to describe the network state more comprehensively, thus providing a broader experimental platform for network security analysis. However, there are various types of network traffic anomalies, and the feature anomalies caused by them will never be reflected in all features. In other words, for the detection of specific abnormal behaviors, not all features are effective, and some features may even cause interference.

At present, the commonly used methods to reduce interference and redundant features mainly include dimensionality reduction algorithms and feature selection algorithms. Dimensionality reduction algorithms reduce the dimensionality of data by converting the feature space into a low-dimensional space, while also reducing irrelevant or redundant features in the dataset. For example, Ref. [24] used dimensionality reduction algorithms to improve the feature space of multiple datasets including KDD, and shows significant improvement in the detection performance for various tasks. The feature selection algorithm is to eliminate interference or redundant features according to certain rules, reduce the number of features, and finally generate an optimal feature set to improve the accuracy of detection. There are many commonly used feature selection methods, and the most effective one is Recursive Feature Elimination with Cross Validation algorithm (RFECV), which is an algorithm based on the Recursive Feature Elimination algorithm (RFE) and introduced cross validation.

Each iteration of the RFECV algorithm includes two stages: the first stage is the basic RFE algorithm, which ranks the importance of features and deletes the least important features; In the second stage, cross validation is introduced to calculate the effect of the new dataset. Finally, according to the results of cross validation, the feature subset with the best performance in the iteration process is automatically selected. However, the RFECV algorithm also has its limitations, that is, the algorithm takes the importance of features as the only basis for feature selection, and deletes the least important features in the process of each iteration, which may result in the loss of some features that are relatively less important but valuable for classification.

To solve the above problems, this section proposes a Greedy RFECV algorithm (GRFECV) based on the RFECV algorithm and the greedy algorithm. In terms of process, GRFECV algorithm, like RFECV, also has the steps of recursively calculating the importance of features

and calculating the cross-check value. But there are great differences between the two in details and ideas. The core idea of the GRFECV algorithm is to take the importance of features as a reference index rather than a selection basis, and then complete each recursive feature selection through cross validation curve. The pseudo code of the GRFECV algorithm is shown in Algorithm 3.

Algorithm 3 The Greedy Recursive Feature Elimination with Cross Validation Algorithm (GRFECV)

Input: data set $X = X_1, X_2, \dots, X_N$, Feature sorter M .

Output: data set $X' = X'_1, X'_2, \dots, X'_k$.

```

1: Define the new feature subset  $X'$  in each iteration,  $X' = X'_1, X'_2, \dots, X'_{new}$ 
2: Define the cross-check value of data set  $X'$  as  $AC(X')$ .
3: Define the loop termination flag Flag and set it to True.
4: while Flag == True do
5:   Define the largest  $AC$  in this round as  $MAX_{value}$ , and the corresponding feature subset as  $MAX_X$ .
6:   Define Host to store the greedy retention features that meet the conditions.
7:   Define  $R_a$  and  $R_b$  to represent the cross-check value of the data set before and after feature elimination.
8:   for index = 1, 2, ..., new - 1 do
9:     Calculate the feature important list  $M(X') = [M_1, M_2, \dots, M_{new+1-index}]$ 
10:    Calculate  $R_a = AC(X')$ .
11:    Delete the least important feature  $M_1$ , and get the feature set  $Z' = Z'_1, Z'_2, \dots, Z'_{new-index}$ .
12:    Calculate  $R_b = AC(Z')$ .
13:    if  $R_a \leq R_b$  then
14:      Deposit  $X'_{index}$  in Host.
15:    else
16:       $MAX_{value} = R_b$ .
17:       $MAX_X = X'_{index+1}, \dots, X'_{new}$ .
18:    end if
19:  end for
20:  Define  $Temp = MAX_X \cup HOST$ .
21:  if  $X' \neq Temp$  then
22:    Update  $X' = Temp$ .
23:  else
24:    Flag = False.
25:  end if
26: end while
27: Return  $X'$ .

```

In the calculation of the GRFECV algorithm, each recursion will get an importance list for all the current features, and then follow the steps of deleting the least important features and calculating the cross-check value according to the order of the features on the table. After traversing all the features in the table, in addition to retaining the local optimal feature subset with the highest cross-check value, all the features that will cause the verification result to decline after deletion are also retained. Then combine these features and perform the above process recursively until the feature table does not change.

We can see that compared with RFECV, which deletes the features with the lowest importance each time and uses the feature importance ranking as the basis for subset generation, the GRFECV algorithm only uses the feature importance list as the reference index, and mainly uses the results of cross-check to filter the features that have a greater impact on the detection effect. Each recursion will re-order the importance of the features selected in the previous round to ensure that the direction of feature selection is the best in the current round.

To sum up, the GRFECV method proposed in this section not only retains the advantages of RFECV in selecting the number of feature subsets and updating the importance of features by cross-checking, but

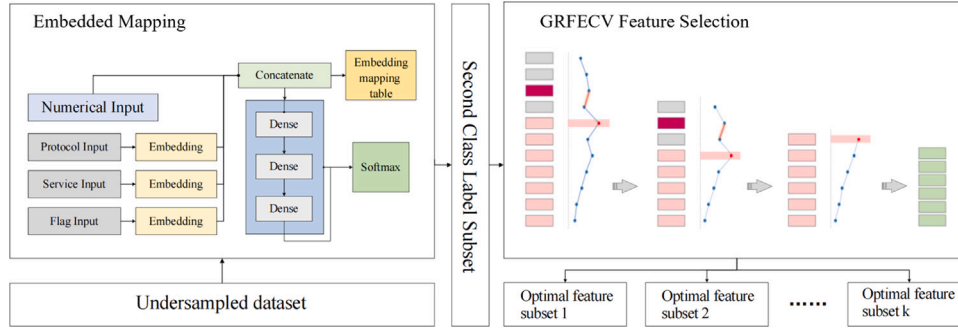


Fig. 6. Overall block diagram of feature extraction algorithm.

also improves the method of feature screening, effectively avoiding the loss of valuable features, so that the obtained feature subset has a stronger detection effect. Because GRFECV retains all the features that have a positive impact on cross checking in each round of recursion, it is called an algorithm affected by greed.

Finally, based on the embedding mapping method of categorical features and the GRFECV algorithm proposed above, this paper proposes a data set feature selection algorithm framework, as shown in Fig. 6.

Firstly, the entity embedding mapping process is performed on the balanced dataset, and the categorical features in the dataset are mapped from sparse unique one-hot codes to low-dimensional dense vectors; Then, the dataset is labeled into several subclasses by labels, and the GRFECV algorithm proposed in this section is used to find the optimal feature subset of each subclass.

2.3. Intrusion detection

2.3.1. A deep neural network model for subclass dataset detection based on GL architecture

Based on the VAGCUS algorithm and GRFECV algorithm proposed above, this section proposes a deep neural network model based on Global and Local (GL), which is used to improve the detection effect of each subclass in the intrusion detection problem. The model is divided into two parts, Global and Local.

The role of the Global layer is to depict the data characteristics as a whole, which can increase the ability of the neural network to represent global traffic. In order to realize the functions of the global layer, this section introduces the Principal Component Analysis (PCA) method to obtain the overall representation of all features of the data.

PCA is a commonly used unsupervised data dimensionality reduction method. Its goal is to find a mapping from high-dimensional space to low-dimensional space, so that the projection of the mapped data in low-dimensional space can be maximumly separable without losing too much information.

In addition, PCA also expects that after the high-dimensional data is mapped, the correlation of each dimension in the low-dimensional space is minimal, and the redundant information between features is minimal. Define a data set $D = \{X_1, X_2, \dots, X_n\}$, X_n represents the n th dimension feature of the data set, then the covariance matrix of X_i is as follows:

$$C(X) = \begin{bmatrix} Var(X_1) & Cov(X_1, X_2) & \dots & Cov(X_1, X_n) \\ Cov(X_2, X_1) & Var(X_2) & \dots & Cov(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ Cov(X_n, X_1) & Cov(X_n, X_2) & \dots & Var(X_n) \end{bmatrix} \quad (13)$$

Where, $Cov(\cdot)$ is the covariance, $Var(\cdot)$ is the variance.

It can be seen that the diagonal value of the covariance matrix is the variance of each dimensional feature, and the other position values are the covariance of each feature. It can be seen from the previous article that the value of the covariance can represent the correlation between the features. When this value is zero, it means that the two features are

not related. Assuming that PCA has realized the mapping of data set X to $Y = \{Y_1, Y_2, \dots, Y_k\}$, and the mapping matrix is P , then:

$$\begin{aligned} C(Y) &= Y^T Y = (X P)^T (X P) \\ &= P^T (X^T X) P \\ &= P^T C(X) P \end{aligned} \quad (14)$$

Therefore, the process of calculating the PCA transformation matrix P is transformed into the process of finding a transformation matrix that can diagonalize the covariance matrix of high-dimensional data sets. The transformation matrix P can be obtained by calculating the eigenvalues and eigenvectors of the covariance matrix $c(X)$.

For the determination of the number of principal components k , we can set an information retention threshold from the perspective of reconstruction and then calculate k . The formula is as follows:

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^n \lambda_i} \geq \delta, \quad (15)$$

where, δ is the threshold, λ_i is the characteristic value of $C(X)$. The data after data balance and feature selection is multiplied by the principal component matrix to obtain the principal component expression of the sample, and participate in the subsequent network construction as the output of the Global layer.

The Local layer is used to extract the deep representation of the optimal feature subset of the target subclass. For example, if the target is to identify Dos attacks, the input of the Local layer is the best feature subset of Dos class extracted by the GRFECV algorithm.

The design idea of the deep neural network model for subclass data set detection based on GL architecture is as follows: First, use Local Net to extract cross feature information from the optimal feature collection subclass through a multi-layer fully connected neural network. Then, splice the output of Local Net and the output of the previous Global layer. Finally, it is trained by the deep neural network and the final classification result is obtained. In this section, the activation function of all middle-layer neurons uses *Relu* function, and the activation function of output neurons uses *Sigmoid* function. The formula of these two activation functions is as follows:

$$Relu(x) = \max(0, x) \quad (16)$$

$$Sigmoid(x) = \frac{1}{1 + e^{-x}}. \quad (17)$$

where x is the input of neural network.

To sum up, the network model based on GL architecture is shown in Fig. 7. The network model can be used for two-class intrusion detection, improve the detection accuracy of each sub-class, and participate in the subsequent overall network construction as a module.

2.3.2. Deep neural network intrusion detection model based on parallel architecture

The previous section proposed a neural network model based on GL architecture, which can output the detection results of a certain subclass, and is a two-class network. If these networks are longitudinally

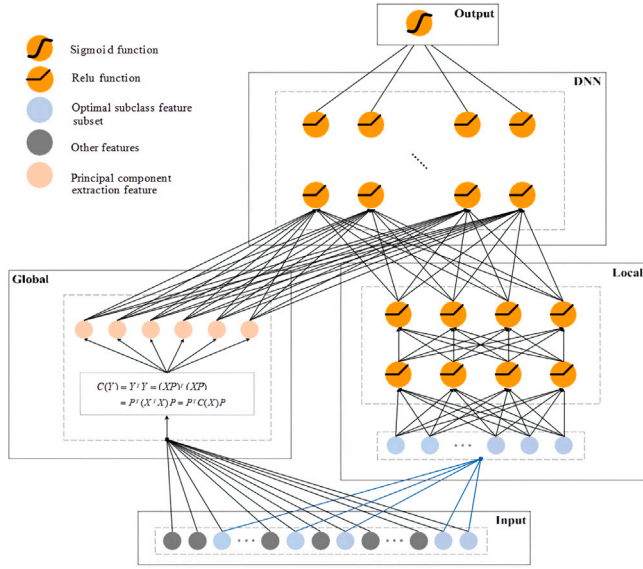


Fig. 7. Subclass detection deep neural network model of GL architecture.

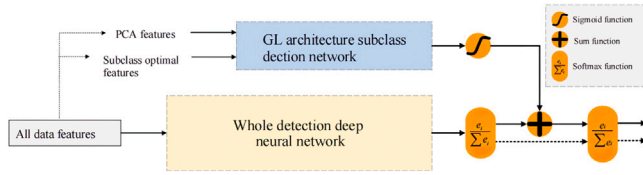


Fig. 8. Adjustment module based on sub-network output.

spliced and all outputs are uniformly processed with *Softmax* activation function, a multi-classification detection model can be obtained. The formula of *Softmax* activation function is as follows:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{c=1}^C e^{z_c}} \quad (18)$$

where C is the number of subclasses in the dataset, z is the input of the activation function *Softmax*.

To improve the detection effect, this section does not directly use the parallel model of these GL subnets as the classification model but proposes a new additive adjustment module based on the output of GL subnets. The construction method of this module is as follows: First, the output of each subnetwork model processed by *Softmax* is regarded as an adjusted value, and the adjusted value is added to the output of a typical multi-classification deep network to obtain an adjusted output probability of this type; After that, the *Softmax* activation function is used to process all the adjustment probabilities to obtain the final output probabilities of each class of samples, thus realizing the multi-classification task. The schematic diagram of the adding and adjusting module is shown in Fig. 8. The output of the adjustment module is as follows:

$$y_i = \text{Softmax}(y_{gl,i} + y_{dnn,i}) = \frac{e^{y_{gl,i} + y_{dnn,i}}}{\sum_{c=1}^C e^{y_{gl,c} + y_{dnn,c}}}, \quad (19)$$

where $y_{gl,i}$ is the i th output of GL Net and $y_{dnn,i}$ is the i th output of the main network.

Finally, based on the above-mentioned GL sub-network adjustment module, this paper proposes a deep neural network model based on the parallel architecture of GL sub-networks. In network training, multi-category cross-entropy is used as the loss function, as shown in the following equation:

$$\text{Loss} = - \sum_{i=1}^n \sum_c \hat{y}_{ic} \cdot \log y_{ic}, \quad (20)$$

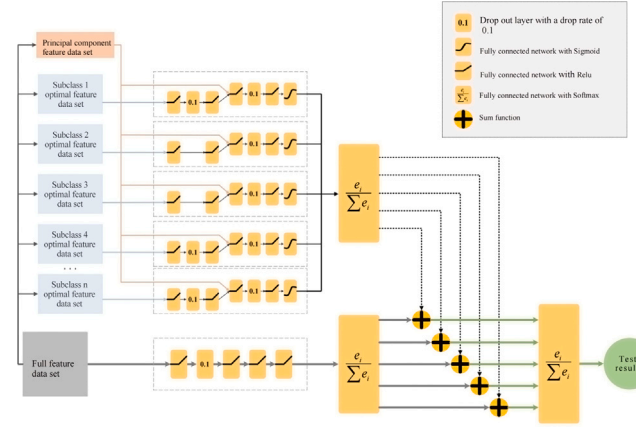


Fig. 9. Intrusion detection deep neural network model under parallel architecture.

Table 1

The quantity information of NSL-KDD data set.

Dataset	Training		Test	
Normal	67343	53.39%	9711	43.08%
DoS	45927	36.65%	7458	33.08%
U2R	52	0.04%	200	0.89%
R2L	995	0.83%	2754	12.22%
Probe	11656	9.09%	2421	10.74%

where n is the number of samples, C is the number of output classes, $\hat{y}$ is the network output, and y is the true label.

After defining the loss function, use the Adam optimizer to update the network parameters to obtain the network parameter combination that minimizes the *Loss*. The calculation steps are as follows:

$$g_t = \frac{\partial \text{Loss}}{\partial \omega_t}, \quad (21)$$

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t, \quad (22)$$

$$V_t = \beta_2 \cdot V_{t-1} + (1 - \beta_2) \cdot g_t^2, \quad (23)$$

$$\omega_{t+1} = \omega_t - lr \cdot \frac{m_t}{1 - \beta_1^t} \cdot \frac{1}{\sqrt{\frac{V_t}{1 - \beta_2^t}}}, \quad (24)$$

where g_t , m_t , and V_t represent respectively the gradient at time t , first-order momentum, and second-order momentum, ω represents the network parameter, lr represents the learning rate, and β represents the modified hyperparameter.

In summary, the corresponding intrusion detection deep neural network model was construct, as shown in Fig. 9.

3. Experiments and analysis

In order to verify the impact of the methods proposed in this paper on the detection effect and the scalability of the proposed methods, all the experiments in this paper take NSL-KDD and CICIDS2017 data sets as the experimental objects.

NSL-KDD data set includes training set and test set, which is an improvement based on KDD99 data set. It deletes redundant records in the training set and duplicate samples in the test set. The number of records from each class is inversely proportional to the percentage of the number of records in the KDD data set. The scale of training set and test set is more reasonable, and the probability distribution of training data and test data is different. The test set contains some attack types that do not appear in the training data set, making the test data set more realistic. The quantity information of NSL-KDD data set is shown in Table 1.

Table 2

The quantity information of CICIDS2017 data set.

Dataset	Training		Test	
Benign	1704522	80.32%	568166	80.33%
Patator	10376	0.49%	3459	0.49%
DoS	284902	13.43%	94837	13.43%
Web	1635	0.08%	545	0.08%
PortScan	119197	5.62%	39733	5.62%
Others	1510	0.07%	503	0.07%

The CICIDS2017 data set contains normal data and several network attacks [42], including Benign (normal data), Web attacks, Brute Force, DoS attacks, DDoS attacks, Hulk, Heartbleed, Botnet, and Port Scan attacks. The data set supports all commonly used network protocols (such as HTTP, HTTPS, FTP, SSH and e-mail protocols), and can reflect the diversity of attacks. Referring to the existing research on CICIDS2017 data set, this paper divides CICIDS2017 data set into six classes: Benign (normal data), Patator (FTP-Patator, SSH Patator), DoS (DoS Hulk, DDoS, DoS GoldenEye, DoS Slowloris, DoS Slowtptest), Web (Web Attack-Brute Force, Web Attack-XSS, Web Attack-Sql Injection), Port Scan, Others (Bot, Infiltration, Heartbleed). In this paper, 75% of the data set is divided into training sets and the remaining 25% is used as test sets by random division. The quantity information of CICIDS2017 data set is shown in Table 2. In addition, there are very few abnormal samples with missing eigenvalues or infinite eigenvalues in CICIDS2017. These abnormal samples will adversely affect the training process, so we delete these samples directly.

In addition, what this paper finally wants to solve is the problem of intrusion detection. Therefore, no matter what problem each algorithm proposed in this paper specifically targets, we should evaluate the advantages and disadvantages of the algorithm based on the detection effect of the data set in the experimental results. Therefore, this experiment uses $F - Measure$ and $Accuracy$ index to evaluate the detection effect of each type of sample. The calculation formula is as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (25)$$

$$F - Measure = \frac{(\alpha^2 + 1)p \cdot r}{\alpha^2 \cdot p + r}, \quad (26)$$

$$p = \frac{TP}{TP + FP},$$

$$r = \frac{TP}{TP + FN},$$

where TP represents the number of positive classes predicted to be positive, TN represents the number of negative classes predicted to be negative, FP represents the number of negative classes predicted to be positive, and FN represents the number of positive classes predicted to be negative. The above indicators are for two-category situations. When it is extended to multi-classification, each target class can be regarded as positive, and the rest classes can be regarded as negative; α is a parameter, p is Precision and r is Recall. In this paper, we select $\alpha = 1$ and get the most common $F1 - Measure$ index. For the convenience of description, we will abbreviate the $Accuracy$ rate as ACC and $F1 - Measure$ as $F1$ in the following content.

In order to evaluate the overall detection effect of the dataset, this paper also introduces Macro-F1-Measure ($M - F1$) and Weighted-F1-Measure ($W - F1$) indicators, with the formula as follows:

$$M - F1 = \frac{1}{c} \sum_{i=1}^c F1_i, \quad (27)$$

$$W - F1 = \sum_{i=1}^c (\omega_i \cdot F1_i), \quad (28)$$

where, c is the number of classes in the dataset, $F1_i$ is the $F1$ value of the class i th sample, ω_i is the weight of the number of class i th sample in the total number of samples in the dataset.

Table 3

Parameter setting table.

Parameters	Values
lr	0.001
β_1	0.9
β_2	0.999
δ	0.95
α	1

The parameters used in the experiment are shown in Table 3. It should be noted that all the experiments in this paper are based on Intel Xeon Gold 5218 Processor and NVIDIA A100 Tensor Core GPU.

3.1. Comparative experiment of detection effect based on VAGCUS method

It can be seen from Tables 1 and 2 that the U2R class and R2L class in NSL-KDD and the Web class, Others class, and Patator class in CICIDS2017 are minority samples, and their number accounts for a small proportion in the corresponding data sets, so these two data sets are extremely unbalanced data sets. Therefore, this section uses the VAGCUS algorithm to balance the data of these two data sets, and through a series of comparative experiments to show the improvement of the VAGCUS undersampling method on the detection effect of NSL-KDD and CICIDS2027, especially for the recognition effect of minority classes. Here, we choose the Random Undersampling algorithm (RUS) and Neighborhood Cleaning Rule algorithm (NCL) as the comparison algorithm. RUS is the simplest and most widely used undersampling algorithm [17]. Its principle is to randomly extract a certain number of majority and minority samples from the data set to form a balanced distributed training data set; NCL is an undersampling algorithm that relies on the nearest neighbor class information to clean data [18]. In addition, this section uses One-hot coding for non-numeric features in the dataset and removes features with values of all zero.

This section sets up several groups of comparative experiments to verify the undersampling method in this paper. The experimental strategy is set as follows:

- (1) The methods involved in the experiment include: the VAGCUS algorithm (VAGCUS-1/2/3) under the three sampling strategies (truncation, boundary reservation, and equal ratio) as the experimental group, and the RUS algorithm, NCL algorithm, and the original data set as the control group, a total of 6 groups of experiments.
- (2) For the NSL-KDD dataset, the target sample size of all majority classes in all undersampling algorithms is set to 2000; For CICIDS2017 data set, the target sample size of majority class Benign is set to 60000, and the target sample size of the majority class Dos and PortScan is set to 20000 in all undersampling algorithms; The number of minority samples in the two data sets will not change.
- (3) SVM, LR, and KNN are used as benchmark classifiers, and the same parameters are used to train these classifiers for each sub-sampling method.
- (4) For each classifier, use the same parameter combination to carry out 20 experiments, and take the average of their results as the final result.

In this section, $F1$ is used to evaluate the detection effect of all classifiers on each subclass of the dataset. The experimental results of NSL-KDD and CICIDS2017 are shown in Tables 4 and 5, respectively. The highest value in each group of comparison indicators is marked in bold, indicating that the corresponding under-sampling method has gained advantages in the comparison of the indicators.

It can be seen from the experimental results that, first of all, whether the experimental object is NSL-KDD or CICIDS2017, for the original dataset without algorithm processing, the detection effect of

Table 4

Subclass F1 score table using different undersampled data sets in NSL-KDD data set.

Algorithm	Class	Classifier		
		SVM	LR	KNN
Raw Datasets	Normal	0.77	0.78	0.77
	DoS	0.89	0.78	0.78
	U2L	0.10	0.13	0.17
	R2L	0.11	0.04	0.04
	Probe	0.71	0.55	0.58
VAGCUS-1	Normal	0.77	0.78	0.78
	DoS	0.88	0.88	0.77
	U2L	0.21	0.18	0.25
	R2L	0.24	0.20	0.11
	Probe	0.71	0.75	0.53
VAGCUS-2	Normal	0.79	0.79	0.78
	DoS	0.88	0.89	0.80
	U2L	0.15	0.14	0.22
	R2L	0.47	0.30	0.14
	Probe	0.72	0.79	0.60
VAGCUS-3	Normal	0.81	0.79	0.80
	DoS	0.89	0.89	0.81
	U2L	0.30	0.32	0.38
	R2L	0.47	0.39	0.22
	Probe	0.77	0.75	0.59
RUS	Normal	0.79	0.78	0.79
	DoS	0.87	0.89	0.82
	U2L	0.21	0.21	0.26
	R2L	0.47	0.30	0.23
	Probe	0.75	0.76	0.60
NCL	Normal	0.77	0.78	0.77
	DoS	0.89	0.78	0.78
	U2L	0.13	0.14	0.20
	R2L	0.10	0.05	0.04
	Probe	0.70	0.56	0.57

the minority classes U2R, R2L, Web, Others, and Patator is very poor on the three classifiers, and the $F1$ index of these minority classes is far lower than the $F1$ value of other classes. It can be seen that the extreme sample imbalance has a great impact on the detection performance of minority samples. After sampling by the sampling algorithm, compared with the original unbalanced data set, the $F1$ indicators of minority samples U2R, R2L, Web, Others, and Patator have improved to varying degrees. It can be considered that for extremely unbalanced data sets, under-sampling the majority class samples will improve the detection performance of minority samples. In addition, the VAGCUS-1/2/3 algorithm obtained 5 and 8 maximum values, NCL obtained 0 and 0 maximum values, and RUS obtained 2 and 1 maximum values in the experimental results of minority samples of two data sets. It can be concluded that the VAGCUS undersampling algorithm proposed in this paper can improve the detection effect of minority samples better than other algorithms.

Secondly, for the majority samples, after processing by VAGCUS undersampling algorithm, the $F1$ value of each majority samples in the experimental results of two data sets has almost improved compared with the original data set, which shows that VAGCUS undersampling algorithm does not lose much original data information. Finally, from the overall point of view of the experiment, we used three classifiers in the experimental results of the two data sets. The NSL-KDD data set has five types of data and obtained 15 groups of experimental results, while CICIDS2017 has six types of data and obtained 18 groups of experimental results, a total of 33 groups. Here we count the winning times of the proposed method. We can see that the VAGCUS-1/2/3 algorithm has won 11 times in the experiment based on the NSL-KDD dataset, 14 times in the experiment based on the CICIDS2017 dataset, and 25 times in 33 groups of evaluation indicators.

Based on the above analysis, we can draw the conclusion that the VAGCUS algorithm, especially the VAGCUS-3 algorithm, has a positive and powerful impact on the detection effect of the data set by

Table 5

Subclass F1 score table using different undersampled data sets in CICIDS2017 data set.

Algorithm	Class	Classifier		
		SVM	LR	KNN
Raw Datasets	Benign	0.75	0.77	0.76
	Patator	0.21	0.16	0.15
	DoS	0.87	0.82	0.80
	Web	0.16	0.17	0.19
	PorScan	0.79	0.69	0.71
	Others	0.13	0.10	0.16
VAGCUS-1	Benign	0.79	0.78	0.75
	Patator	0.29	0.26	0.33
	DoS	0.87	0.89	0.83
	Web	0.25	0.27	0.20
	PorScan	0.79	0.75	0.73
	Others	0.21	0.19	0.19
VAGCUS-2	Benign	0.76	0.80	0.75
	Patator	0.26	0.26	0.19
	DoS	0.88	0.82	0.79
	Web	0.24	0.20	0.23
	PorScan	0.77	0.72	0.78
	Others	0.25	0.18	0.21
VAGCUS-3	Benign	0.77	0.77	0.82
	Patator	0.43	0.36	0.37
	DoS	0.89	0.84	0.80
	Web	0.39	0.33	0.35
	PorScan	0.80	0.80	0.78
	Others	0.37	0.29	0.31
RUS	Benign	0.77	0.80	0.73
	Patator	0.40	0.37	0.36
	DoS	0.86	0.83	0.85
	Web	0.25	0.28	0.23
	PorScan	0.82	0.73	0.79
	Others	0.23	0.14	0.17
NCL	Benign	0.76	0.73	0.79
	Patator	0.21	0.19	0.19
	DoS	0.87	0.88	0.83
	Web	0.18	0.20	0.17
	PorScan	0.79	0.79	0.76
	Others	0.15	0.21	0.19

processing the unbalanced data set. The experimental results strongly prove that the VAGCUS undersampling algorithm achieves the expected effect in processing extremely unbalanced data sets.

3.2. Optimal feature subset selection experiment

The experiments in this section take NSL-KDD and CICIDS2017 datasets that have been under-sampled by the VAGCUS algorithm as the experimental objects. The purpose of the experiment is to verify the improvement of the GRFECV feature selection algorithm proposed in this paper on the detection effect of each sub-class sample in the dataset. The number of samples in the two data sets is the same as in the previous experiment.

First, we use entity embedding maps to handle non-numeric features in NSL-KDD, resulting in 52-dimensional features. Since there are no non-digital features in CICIDS2017, we only need to delete its IP and timestamp and other features that are not related to detection, and finally get 78-dimensional features, and then we number these features. The feature name corresponding to the feature number (Fn) in the tables can be found in the feature table of APPENDIX I and APPENDIX II. Finally, we use the RFECV algorithm and GRFECV algorithm to feature select the NSL-KDD data set to obtain the optimal feature subset as shown in Tables 6 and 7, and use the same algorithm to feature select the CICIDS2017 data set to obtain the optimal feature subset as shown in Tables 8 and 9.

Table 6

RFECV optimal feature subsets of each subclass in NSL-KDD Dataset.

Subclass	Quality	Feature number(Fn)
Normal	38	f0, f1, f3–f13, f16, f17, f19, f21–f23, f25, f28, f30, f32–f34, f36, f39–f50
DoS	16	f1, f3, f4, f5, f9, f12, f14, f16, f23, f24, f33, f35, f40, f48, f50
U2R	12	f6, f10, f17, f24, f25, f28, f32, f42, f45, f47, f48
R2L	28	f0, f3, f4, f7, f9, f10, f16, f17, f21–f23, f25, f27–f30, f33, f34, f42–f51
Probe	22	f1, f3, f5, f6, f7, f11, f12, f14, f16, f17, f19, f33, f35, f37, f40, f43–f48, f50

Table 7

GRFECV optimal feature subsets of each subclass in NSL-KDD Dataset.

Subclass	Quality	Feature number(Fn)
Normal	30	f0, f3, f5–f10, f12, f16, f17, f19, f21–f23, f25, f28, f30, f32, f33, f39, f40, f42–f49
DoS	19	f1, f4, f5, f9, f12, f14, f16, f17, f19, f23, f24, f33–f35, f40, f45, f46, f48, f50
U2R	19	f0, f1, f3, f5, f6, f8, f9, f10, f16, f17, f23, f25, f27, f28, f32, f33, f40, f42, f45
R2L	28	f0, f3, f4, f6, f10, f12, f16, f17, f21–f25, f27, f29, f30, f32–f34, f42, f44–f51
Probe	22	f1, f5, f7, f8, f11, f12, f14, f16, f17, f19, f33, f35, f37, f39, f40, f43–f48, f50

Table 8

RFECV optimal feature subsets of each subclass in CICIDS2017 Dataset.

Subclass	Quality	Feature number(Fn)
Benign	58	f0–f29, f34–f43, f46–f48, f50, f51, f65–f77
Patator	48	f0–f10, f14–f32, f34–f37, f39–f41, f46–f48, f51, f52, f66–f69, f71, f76
DoS	58	f0–f29, f33–f37, f39–f43, f46–f48, f51, f52, f64, f66–f77
Web	52	f0–f27, f34–f39, f46–f48, f50, f51, f65–f77
PortScan	56	f0–f30, f34–f42, f46–f48, f51, f52, f54, f66–f70, f72–f74, f76, f77
Others	48	f0–f27, f35–f40, f46, f48, f66–f70, f73–f77

Table 9

GRFECV optimal feature subsets of each subclass in CICIDS2017 Dataset.

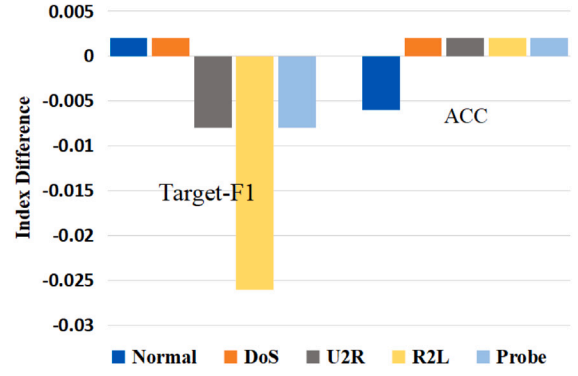
Subclass	Quality	Feature number(Fn)
Benign	50	f0, f2–f29, f34–f43, f51–f53, f66–f75
Patator	48	f0–f11, f14–f30, f32, f34–f37, f39–f41, f46–f48, f51, f52, f66–f69, f71, f76
DoS	55	f0–f30, f38–f43, f46–f48, f51, f52, f64, f66–f77
Web	59	f0–f30, f34–f43, f46–f48, f51, f52, f64, f66–f77
PortScan	52	f0–f30, f34–f43, f50–f52, f66–f70, f72–f74, f76, f77
Others	58	f0–f30, f34–f43, f46, f48, f51, f52, f64, f66–f77

3.2.1. Comparative experiment of GRFECV and RFECV

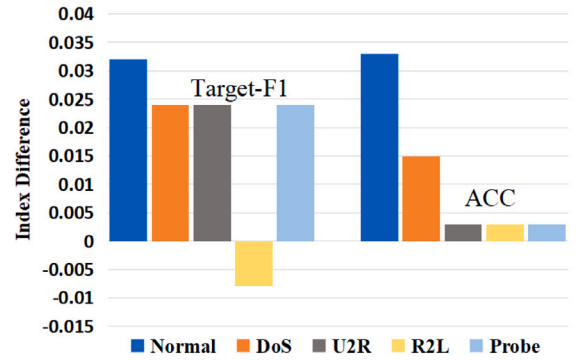
After obtaining the optimal feature subset of each class of samples in the dataset, we take the NSL-KDD dataset as an example, and then use the corresponding optimal feature subset to test the detection effect of the subclasses, and compare the performance of GRFECV and RFECV algorithms according to the detection effect, to verify the improvement of the GRFECV feature selection algorithm proposed in this paper.

In order to compare the influence of feature selection algorithm on the detection effect of each subclass, this section sets the experimental strategy as follows:

- (1) Classify the multi-classified data set into five sub-data sets. In each dataset, the label of the corresponding class is set to 1, marked as the target class, and the label of the other classes is set to 0.
- (2) Select XGboost, MLP and DT as validation classifiers to test the classification effect.
- (3) The classification effect is evaluated using the F1 index and the overall ACC index of the target class.



(a) RFECV each subclass detection effect



(b) GRFECV each subclass detection effect

Fig. 10. Performance comparison of GRFECV and RFECV under XGBoost classifier.

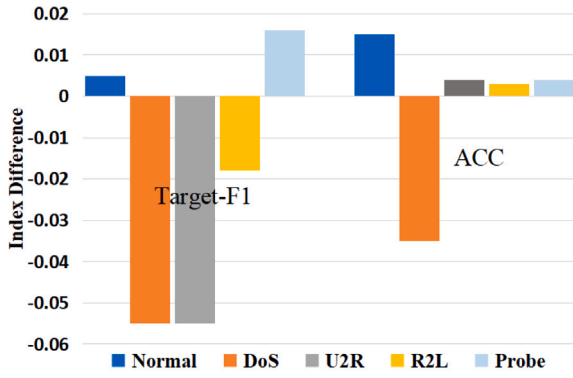
- (4) For each classifier, the same parameters are used for 20 experiments, and the average value of 20 indicators is taken as the final result.

This section transforms the original multi-classification problem into several binary classification problems to study, but the binary classification test results with multi-classification as the goal are extremely high. In order to more clearly show the impact of feature selection on classification performance and the gap between RFECV and GRFECV, we express the result as the difference between the test index obtained using the best feature subset and the test index obtained using the data set with all features. The results are shown in Figs. 10 to 12.

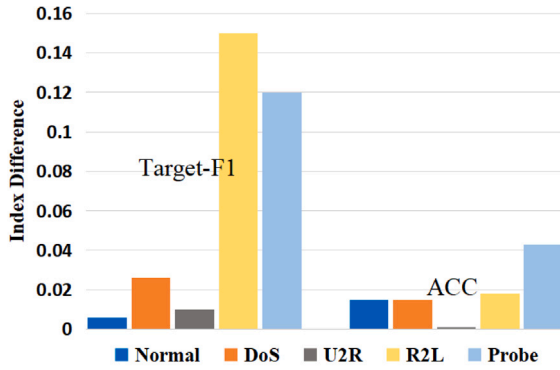
Now the results of the experiment are analyzed. It is known that there are 3 types of classifiers, 5 sets of subclass data, 2 types of index parameters, and a total of 30 indexes. First, analyze the performance improvement of GRFECV relative to the full feature data set. Among the 30 indicators of GRFECV, only 1 item is below the horizontal line, and 29 items are above the horizontal line, and GRFECV has greatly improved the detection effect of almost every subclass. This shows that the GRFECV feature extraction algorithm can well improve the detection effect of each subclass.

Then compare the GRFECV and RFECV algorithms, after using RFECV to extract features, a total of 11 of the 30 indicators are below the horizontal line, and only 19 are above the horizontal line. From the experimental results, we can easily see that in most cases, the detection effect of each subclass after RFECV processing is not greatly improved, and even a considerable part of the subclass detection effect is lower than that without RFECV processing. This shows that the performance of RFECV is much lower than that of the GRFECV.

In summary, the GRFECV algorithm proposed in this paper has a huge improvement over the original RFECV algorithm, and can effectively improve the model's detection effect for each subclass.

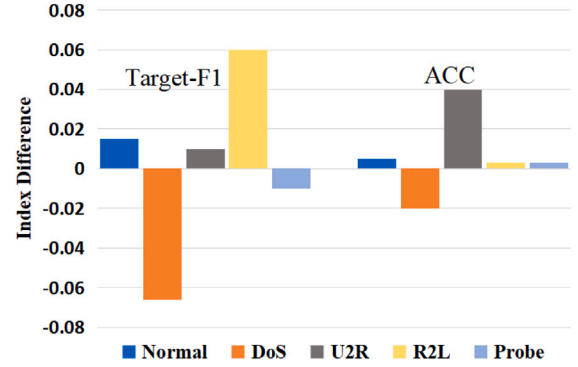


(a) RFECV each subclass detection effect

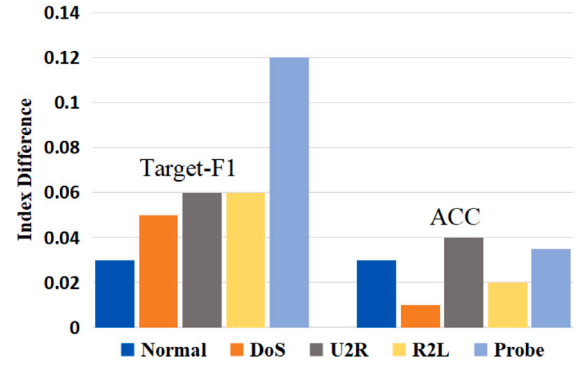


(b) GRFECV each subclass detection effect

Fig. 11. Performance comparison of GRFECV and RFECV under MLP classifier.



(a) RFECV each subclass detection effect



(b) GRFECV each subclass detection effect

Fig. 12. Performance comparison of GRFECV and RFECV under DT classifier.

3.2.2. Comparative experiment between GRFECV and other feature selection algorithms

This section also takes the NSL-KDD data set and CICIDS2017 data set as experimental objects and compares the GRFECV algorithm with the current popular Max-Relevance and Min-Redundancy (MRMR) and Mutual Information Maximization (MIM) feature selection algorithms to verify the effect of GRFECV. The principle of MRMR is to find a group of features in the original feature set that has the greatest correlation with the final output result but the least correlation between features [22]. The principle of MIM is to calculate the mutual information between each feature and category and sort them according to the size of mutual information, and then select a specified number of features for feature selection [23]. This section uses experimental conditions that are completely consistent with previous experiments, the evaluation index is $F1$, and the classifier uses XGBoost (XGB) and MLP. It should be noted that these algorithms, like feature selection methods based on feature importance, also need to manually specify the number of features. For the accuracy of the experiment, this section selects the same number of features as the GRFECV method for them.

To more intuitively show the performance difference between various feature selection methods and the GRFECV proposed in this paper, we let the detection performance indicators obtained by using other methods subtract the detection performance indicators obtained by using GRFECV, and the difference is shown in Tables 10 and 11. The vertical axis in the figure is the difference between the detection indicators of other algorithms and the GRFECV algorithm. When the value is negative, the GRFECV algorithm performs better.

It can be seen from the table that for the detection results of each subclass of the two data sets, no matter which subclass and classifier, the indicators of the two feature selection methods can hardly reach the level of GRFECV.

Table 10

Differences of $F1$ between other feature selection methods and GRFECV performance indexes in NSL-KDD dataset.

	XGB		MLP	
	MRMR	MIM	MRMR	MIM
Normal	-0.06	-0.05	0	0
DoS	-0.06	-0.02	-0.16	-0.17
U2R	-0.01	-0.01	-0.04	-0.08
R2L	0.02	-0.04	0.02	-0.24
Probe	-0.02	-0.12	-0.11	-0.27

Table 11

Differences of $F1$ between other feature selection methods and GRFECV performance indexes in CICIDS2017 dataset.

	XGB		MLP	
	MRMR	MIM	MRMR	MIM
Benign	-0.05	-0.04	-0.01	-0.11
Patator	-0.02	-0.01	-0.05	0.08
DoS	-0.16	-0.04	-0.12	-0.01
Web	0.01	-0.02	-0.06	-0.08
PostScan	-0.14	-0.03	-0.01	0
Others	-0.01	0.02	-0.07	-0.08

From the overall point of view, in the experimental results based on the NSL-KDD data set, 5 subclasses, 2 classifiers, 2 groups of comparison algorithms, and 1 index parameter, a total of 20 comparison results, of which only 2 are slightly higher than GRFECV and 2 are equal to GRFECV; In the experimental results based on CICIDS2017 data set, 6 subclasses, 2 classifiers, 2 groups of comparison algorithms, and 1 index parameter, a total of 24 comparison results, of which only 3 are slightly

Table 12

Improved effect Table of GL SUBNET Parallel Deep Neural Network for NSL-KDD Data Set (Classification comprehensive index).

Index parameters	CGL-DNN	DNN	Local Net	GL Net
Normal-F1	0.87	0.81	0.81	0.82
Dos-F1	0.90	0.86	0.83	0.80
U2R-F1	0.22	0.15	0	0.24
R2L-F1	0.45	0.48	0.33	0.42
Probe-F1	0.85	0.72	0.66	0.76
M-F1	0.66	0.61	0.53	0.61
W-F1	0.87	0.72	0.66	0.76
ACC	0.83	0.78	0.76	0.77

higher than GRFECV and 1 are equal to GRFECV. It can be seen that under the current comparative experiment strategy, the overall effect of these methods is not as good as GRFECV.

In conclusion, the experiment we designed not only verified the improvement of the GRFECV feature selection algorithm compared with the RFECV algorithm but also proved that GRFECV performance is better than the other two popular feature selection algorithms with the same number of features. Finally, this section applies the optimal feature subset of the two data sets obtained by the GRFECV algorithm to the subsequent experiments.

3.3. Intrusion detection experiment of deep neural network

This section will verify the overall effect of multi-classification detection of the deep neural network intrusion detection algorithm based on the GL subnet parallel architecture. The experimental strategy is as follows.

- (1) This experiment is conducted based on previous experiments. The data set used is the NSL-KDD data set and CICIDS2017 data set processed by the VAGCUS undersampling algorithm and GRFECV feature selection algorithm. In the two data sets processed by the algorithm, the number of samples of each subclass and the setting of the optimal feature subset of each subclass are the same as in the previous experiments.
- (2) The construction of the model is based on TensorFlow 2.4.0.
- (3) The contrast experiment is divided into two groups: the model proposed in this paper is compared with the same parameter deep network model without GL subnet parallel architecture to show the improvement effect; Compared with other commonly used machine learning methods and deep learning methods to show the performance level.
- (4) This section mainly tests the overall detection performance of the method, so the index selects $F1$ value and the overall $M - F1$ value, $W - F1$ value, and ACC value.
- (5) For each classifier, use the same parameters for 20 experiments, and take the average of their results as the final result.

3.3.1. Comparative experiment between the improved model and the original model

First, we verify the improvement effect of the deep neural network model based on the GL subnet parallel architecture (CGL-DNN). The comparison items include: deep neural network without GL subnet architecture (DNN), feature selection network based on GL subnet module (Local Net), and GL Net. DNN is the baseline model, Local Net is an improved model combined with the GRFECV feature selection algorithm, and GL Net is a subclass detection network model based on this model. The model parameters that are selected for the three groups of comparative experiments are completely consistent with CGL-DNN. The experimental results of the two data sets are shown in Tables 12 and 13 below, and the maximum value of each index in the table will be marked in bold.

Table 13

Improved effect Table of GL SUBNET Parallel Deep Neural Network for CICIDS2017 Data Set (Classification comprehensive index).

Index parameters	CGL-DNN	DNN	Local Net	GL Net
Benign-F1	0.86	0.76	0.75	0.72
Patator-F1	0.40	0.35	0.33	0.41
DoS-F1	0.91	0.87	0.85	0.79
Web-F1	0.33	0.33	0.29	0.35
PostScan-F1	0.80	0.75	0.72	0.74
Others-F1	0.29	0.32	0.28	0.29
M-F1	0.60	0.56	0.54	0.55
W-F1	0.86	0.79	0.79	0.76
ACC	0.85	0.81	0.78	0.80

Table 14

The effect of CGL-DNN compared with other machine learning and deep learning models on NSL-KDD data set.

Index parameters	CGL-DNN	SVM	CNN	AEL
Normal-F1	0.87	0.80	0.79	0.80
Dos-F1	0.90	0.89	0.85	0.56
U2R-F1	0.22	0.23	0.27	0
R2L-F1	0.45	0.45	0.25	0.19
Probe-F1	0.85	0.75	0.69	0.22
M-F1	0.66	0.62	0.57	0.35
W-F1	0.88	0.82	0.80	0.65
ACC	0.83	0.78	0.77	0.62

Table 15

The effect of CGL-DNN compared with other machine learning and deep learning models on CICIDS2017 data set.

Index parameters	CGL-DNN	SVM	CNN	AEL
Benign-F1	0.86	0.78	0.79	0.81
Patator-F1	0.40	0.39	0.34	0.34
DoS-F1	0.91	0.88	0.83	0.50
Web-F1	0.33	0.31	0.34	0.29
PostScan-F1	0.80	0.81	0.73	0.69
Others-F1	0.29	0.30	0.32	0.19
M-F1	0.60	0.58	0.56	0.47
W-F1	0.86	0.79	0.79	0.76
ACC	0.84	0.80	0.77	0.59

From the data in the table, whether based on NSL-KDD or CICIDS2017, first of all, the effect of Local Net and GL Net is almost gradually enhanced, which shows that the improvement idea of the model based on Global and Local has achieved success. Then, CGL-DNN has significantly improved compared with GL Net and DNN in terms of both subclass detection effect and overall detection effect.

To sum up, the improvement effect of the deep neural network model based on GL sub-network parallel architecture proposed in this paper is in line with expectations.

3.3.2. Comparative experiment with other detection models

Now we compare the detection performance of the CGL-DNN algorithm with other machine learning and deep learning algorithms. The machine learning algorithm selects SVM, and the deep learning algorithm selects the current popular CNN. In addition, it also adds a comparison with AutoEncoder combined with the LR classifier framework (AELR). The experimental results of the two data sets are shown in Tables 14 and 15. The maximum value of each index is also marked in bold.

It can be seen that the CGL-DNN algorithm proposed in this paper has greatly improved compared with the traditional machine learning algorithm SVM in the experimental results of the two data sets, and its overall accuracy ACC is 0.05 and 0.04 higher than that of the SVM algorithm. The deep learning algorithm CNN performs poorly, with almost all its indicators lower than CGL-DNN, but it is better than AELR. The AELR model performs the worst in the two groups

Table 16

Table of standard deviation of index parameters for NSL-KDD data set.

Index parameters	Standard deviation	Index parameters	Standard deviation
Normal-F1	4.19%	R2L-F1	4.91%
DoS-F1	3.94%	Probe-F1	4.07%
U2R-F1	5.13%	ACC	3.83%

Table 17

Table of standard deviation of index parameters for CICIDS2017 data set.

Index parameters	Standard deviation	Index parameters	Standard deviation
Benign-F1	4.51%	PortScan-F1	4.65%
Pator-F1	5.25%	Others-F1	5.15%
DoS-F1	4.73%	ACC	4.16%
Web-F1	4.95%		

Table 18

All features and feature numbers (Fn) of NSL-KDD.

Fn	Feature name	Fn	Feature name
f1	duration	f27	su attempted
f2	Eprotocol type0	f28	num root
f3	Eprotocol type1	f29	num file creations
f4	Eservice0	f30	num shells
f5	Eservice1	f31	num access files
f6	Eservice2	f32	is host login
f7	Eservice3	f33	is guess login
f8	Eservice4	f34	count
f9	Eservice5	f35	srv count
f10	Eservice6	f36	serror rate
f11	Eservice7	f37	srv rerror rate
f12	Eservice8	f38	rerror rate
f12	Eflag0	f39	srv rerror rate
f14	Eflag1	f40	same srv rate
f15	Eflag2	f41	diff srv rate
f16	Eflag3	f42	srv diff host rate
f17	src bytes	f43	dh counft ^a
f18	dst bytes	f44	dh srv count
f19	land	f45	dh same srv rate
f20	wrong fragment	f46	dh diff srv rate
f21	urgent	f47	dh same src port rate
f22	hot	f48	dh srv diff host rate
f23	num failed logins	f49	dh serror rate
f24	logged in	f50	dh srv error rate
f25	num compromised	f51	dh rerror rate
f26	root shell	f52	dh srv rerror rate

^a The meaning of dh and dst host is the same.

of experimental results, which also proves that the effect will not be ideal if the deep learning algorithm in other fields is directly transferred without targeted improvement.

In addition, we calculated the standard deviation of the corresponding index according to the results of 20 experiments, and obtained Tables 16 and 17. In the experiment based on NSL-KDD data set, the standard deviation of *F1* and *ACC* for subclass detection almost does not exceed 5%; In the experiment based on CICIDS2017 data set, the standard deviation of *F1* and *ACC* is about 5%. It can be proved that the detection model proposed in this paper has good detection stability.

To sum up, the deep neural network intrusion detection algorithm (CGL-DNN) based on GL subnet parallel proposed in this paper has a great improvement over the original DNN, and also maintains advantages over other commonly used machine learning and deep learning algorithms. The algorithm proposed in this paper can provide a targeted and effective solution to the intrusion detection for IIOT.

3.4. Some discussions on the scalability and complexity of algorithms

From the experimental results of this paper, it is not difficult to see that all the algorithms and models proposed in this paper have greatly

improved the detection effect of NSL-KDD data set and CICIDS2017 data set, which also shows that the algorithms and models proposed in this paper have good scalability and can be effectively applied to the detection of other data sets. In fact, the algorithm proposed in this paper is aimed at the problem of sample imbalance, feature redundancy and interference of the dataset. Therefore, in theory, as long as the dataset has these problems, the method proposed in this paper can be used to deal with them, so as to improve the detection effect of the dataset.

From the perspective of algorithm complexity, for Algorithm 2, its complexity in the serial setting is $O(n^3)$. If we set it to run in a parallel environment, and perform undersampling operations on all subclasses in the dataset at the same time, then the complexity of Algorithm 2 is $O(n^2)$; For Algorithm 3, its input is a single subclass dataset in the dataset, so its complexity is $O(n^2)$ in both serial and parallel settings. However, whether it is the undersampling algorithm (Algorithm 2) or the feature extraction algorithm (Algorithm 3), the parallel setting will improve the efficiency of data processing in the experiment. The experiments in this paper are all carried out in serial settings.

3.5. Future work

In view of the extreme imbalance between the number of samples in the majority class and the number of samples in the minority class in the data set, we have conducted a lot of research on the undersampling algorithm for the majority class data, and proposed the VAGCUS algorithm and the data balancing algorithm framework based on the VAGCUS algorithm (Fig. 4). However, there is not much research on oversampling algorithm (data generation algorithm) for the minority class data and its method combined with VAGCUS. In fact, in some cases, in order to obtain a more reasonable and fully balanced data set, it is not enough to rely only on the undersampling algorithm for the majority class samples. Sometimes it is also necessary to consider oversampling the minority class data samples, which is one of the purposes of the data balancing algorithm framework proposed in this paper. At the same time, in recent years, researchers have become increasingly enthusiastic about new data generation algorithms such as generative adversarial networks, and combining these algorithms with the VAGCUS algorithm may produce interesting results. Therefore, in the follow-up work, we will combine the VAGCUS algorithm with the data generation algorithm, that is, conduct in-depth research on the data balancing algorithm framework proposed in this paper to obtain a more reasonable balanced data set.

In addition, some samples in the data set may have abnormal conditions such as missing eigenvalues or infinite eigenvalues. For example, there are a small number of abnormal samples in the CICIDS2017 data set, which will adversely affect the training process. When the number of abnormal samples is very small, we can delete them directly, but as the number of abnormal samples increases, directly deleting them will reduce the detection effect. Therefore, how to better deal with these abnormal samples is also one of the issues to be studied in the follow-up work.

Finally, with the rapid development of IIOT, IIOT will face more new types of network attacks. Therefore, in the follow-up work, our experiments can no longer be limited to only using public data sets, and we can also consider using real-time network data captured by network traffic packet capture tools, which is more practical.

4. Conclusions

In view of the extreme imbalance in the number of samples of different classes in the dataset, redundant and meaningless features in the samples, and the inability of traditional intrusion detection methods to meet the requirements of detection accuracy in the increasingly complex IIOT, this paper proposes three solutions. Firstly, to address the problem of sample imbalance, this paper proposes an under-sampling

Table 19

All features and feature numbers (Fn) of CICIDS2017.

Fn	Feature name	Fn	Feature name
f1	Destination Port	f40	Max Packet Length
f2	Flow Duration	f41	Packet Length Mean
f3	Total Fwd Packets	f42	Packet Length Std
f4	TB Packets	f43	Packet Length Variance
f5	TL of Fwd Packets	f44	FIN Flag Count
f6	TL of Bwd Packets	f45	SYN Flag Count
f7	Fwd PL Max	f46	RST Flag Count
f8	Fwd PL Min	f47	PSH Flag Count
f9	Fwd PL Mean	f48	ACK Flag Count
f10	Fwd PL Std	f49	URG Flag Count
f11	Bwd PL Max	f50	CWE Flag Count
f12	Bwd PL Min	f51	ECE Flag Count
f13	Bwd PL Mean	f52	Down/Up Ratio
f14	Bwd PL Std	f53	Average Packet Size
f15	Flow Bytes/s	f54	Avg Fwd Segment Size
f16	Flow Packets/s	f55	Avg Bwd Segment Size
f17	Flow IAT Mean	f56	Fwd Header Length
f18	Flow IAT Std	f57	Fwd Avg Bytes/Bulk
f19	Flow IAT Max	f58	Fwd Avg Packets/Bulk
f20	Flow IAT Min	f59	Fwd Avg Bulk Rate
f21	Fwd IAT Total	f60	Bwd Avg Bytes/Bulk
f22	Fwd IAT Mean	f61	Bwd Avg Packets/Bulk
f23	Fwd IAT Std	f62	Bwd Avg Bulk Rate
f24	Fwd IAT Max	f63	Subflow Fwd Packets
f25	Fwd IAT Min	f64	Subflow Fwd Bytes
f26	Bwd IAT Total	f65	Subflow Bwd Packets
f27	Bwd IAT Mean	f66	Subflow Bwd Bytes
f28	Bwd IAT Std	f67	Init Win bytes forward
f29	Bwd IAT Max	f68	Init Win bytes backward
f30	Bwd IAT Min	f69	act data pkt fwd
f31	Fwd PSH Flags	f70	min seg size forward
f32	Bwd PSH Flags	f71	Active Mean
f33	Fwd URG Flags	f72	Active Std
f34	Bwd URG Flags	f73	Active Max
f35	Fwd Header Length	f74	Active Min
f36	Bwd Header Length	f75	Idle Mean
f37	Fwd Packets/s	f76	Idle Std
f38	Bwd Packets/s	f77	Idle Max
f39	Min Packet Length	f78	Idle Min

algorithm based on hierarchical clustering. This algorithm can reduce the number of majority samples while maintaining the information diversity of majority samples, thus effectively alleviating the sample imbalance and inhibiting the over-fitting phenomenon of majority samples. Secondly, to address feature redundancy and interference, this paper proposes a GRFECV optimal feature selection algorithm. This algorithm can obtain the optimal feature subset of each subclass in the dataset and eliminate redundant and interfering features. Thirdly, to apply deep learning to the field of intrusion detection more reasonably, this paper proposes a deep neural network intrusion detection model based on the parallel connection of global and local subnetworks. This model successfully applies deep learning to the intrusion detection technology and realizes the construction of the intrusion detection classifier for IIOT based on deep learning, thus greatly improving the effect of the intrusion detection for IIOT. Finally, corresponding simulation experiments are designed to verify the effectiveness and superiority of the proposed algorithm and model.

CRedit authorship contribution statement

Yaoyao Lu: Conceptualization, Methodology, Software, Writing – original draft. **Senchun Chai:** Review & editing, supervision. **Yuhan Suo:** Methodology, Writing – review & editing. **Fenxi Yao:** Review. **Chen Zhang:** Review.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Appendix A. Table 18: All features and feature numbers of NSL-KDD

See Table 18.

Appendix B. Table 19: All features and feature numbers (Fn) of CICIDS2017

See Table 19.

References

- [1] M. Peña-Cabrera, V. Lomas, G. Lefranc, Fourth industrial revolution and its impact on society, in: 2019 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies, CHILECON, IEEE, 2019, pp. 1–6.
- [2] S. Pal, Z. Jadidi, Analysis of security issues and countermeasures for the industrial internet of things, *Appl. Sci.* 11 (20) (2021) 9393.
- [3] V. Mullet, P. Soni, E. Ramat, A review of cybersecurity guidelines for manufacturing factories in industry 4.0, *IEEE Access* 9 (2021) 23235–23263.
- [4] A. Haleem, M. Javaid, R.P. Singh, S. Rab, R. Suman, Perspectives of cybersecurity for ameliorative Industry 4.0 era: A review-based framework, *Ind. Robot. Int. J. Robot. Res. Appl.* (2022).
- [5] A. Heidari, M.A. Jabrael Jamali, Internet of Things intrusion detection systems: A comprehensive review and future directions, *Cluster Comput.* (2022) 1–28.
- [6] N. Karnik, U. Bora, K. Bhadri, P. Kadambi, P. Dhattrak, A comprehensive study on current and future trends towards the characteristics and enablers of industry 4.0, *J. Ind. Inf. Integr.* 27 (2022) 100294.
- [7] T. Alladi, V. Chamola, S. Zeadally, Industrial control systems: Cyberattack trends and countermeasures, *Comput. Commun.* 155 (2020) 1–8.
- [8] I. Mistry, S. Tanwar, S. Tyagi, N. Kumar, Blockchain for 5G-enabled IoT for industrial automation: A systematic review, solutions, and challenges, *Mech. Syst. Signal Process.* 135 (2020) 106382.
- [9] A.A. Mirani, G. Velasco-Hernandez, A. Awasthi, J. Walsh, Key challenges and emerging technologies in industrial IoT architectures: A review, *Sensors* 22 (15) (2022) 5836.
- [10] X. Zhang, X. Cai, C. Wang, K. Han, S. Zhang, A dynamic security control architecture for industrial cyber-physical system, in: 2019 IEEE International Conference on Industrial Internet, ICII, IEEE, 2019, pp. 148–151.
- [11] N. Koroniotis, N. Moustafa, E. Sitnikova, A new network forensic framework based on deep learning for Internet of Things networks: A particle deep framework, *Future Gener. Comput. Syst.* 110 (2020) 91–106.
- [12] T.-T.-H. Le, Y.E. Oktian, H. Kim, XGBoost for imbalanced multiclass classification-based industrial internet of things intrusion detection systems, *Sustainability* 14 (14) (2022) 8707.
- [13] M. Sainz, I. Garitano, M. Iturbe, U. Zurutuza, Deep packet inspection for intelligent intrusion detection in software-defined industrial networks: A proof of concept, *Logic J. IGPL* 28 (4) (2020) 461–472.
- [14] Q. Niyaz, W. Sun, A.Y. Javaid, A deep learning based ddos detection system in software-defined networking (SDN), 2016, arXiv preprint arXiv:1611.07400.
- [15] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: 2015 Military Communications and Information Systems Conference, MilCIS, IEEE, 2015, pp. 1–6.
- [16] Y. Suo, R. Wang, S. Chai, R. Chai, M. Su, NKB-S: Network intrusion detection based on SMOTE sample generation, in: Cognitive Systems and Information Processing: 7th International Conference, ICCSIP 2022, Fuzhou, China, December 17–18, 2022, Revised Selected Papers, Springer, 2023, pp. 130–147.
- [17] B. Liu, G. Tsoumakas, Dealing with class imbalance in classifier chains via random undersampling, *Knowl.-Based Syst.* 192 (2020) 105292.
- [18] K. Agustianto, P. Destarianto, Imbalance data handling using neighborhood cleaning rule (NCL) sampling method for precision student modeling, in: 2019 International Conference on Computer Science, Information Technology, and Electrical Engineering, ICOMITEE, IEEE, 2019, pp. 86–89.
- [19] M. Farajzadeh-Zanjani, E. Hallaji, R. Razavi-Far, M. Saif, Generative-adversarial class-imbalance learning for classifying cyber-attacks and faults-a cyber-physical power system, *IEEE Trans. Dependable Secure Comput.* 19 (6) (2021) 4068–4081.
- [20] N. Gupta, V. Jindal, P. Bedi, LIO-IDS: Handling class imbalance using LSTM and improved one-vs-one technique in intrusion detection system, *Comput. Netw.* 192 (2021) 108076.

- [21] M.S. Al-Daweri, K.A. Zainol Ariffin, S. Abdullah, M.F.E. Md. Senan, An analysis of the KDD99 and UNSW-NB15 datasets for the intrusion detection system, *Symmetry* 12 (10) (2020) 1666.
- [22] A. Samat, E. Li, P. Du, S. Liu, J. Xia, GPU-accelerated CatBoost-forest for hyperspectral image classification via parallelized mRMR ensemble subspace feature selection, *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* 14 (2021) 3200–3214.
- [23] M. Tschannen, J. Djolonga, P.K. Rubenstein, S. Gelly, M. Lucic, On mutual information maximization for representation learning, 2019, arXiv preprint arXiv:1907.13625.
- [24] E. Hallaji, M. Farajzadeh-Zanjani, R. Razavi-Far, V. Palade, M. Saif, Constrained generative adversarial learning for dimensionality reduction, *IEEE Trans. Knowl. Data Eng.* (2021).
- [25] E. Hallaji, R. Razavi-Far, M. Saif, Enhancing detection accuracy of cyber attacks through dimensionality reduction, in: 30th European Safety and Reliability Conference, ESREL 2020 and 15th Probabilistic Safety Assessment and Management Conference, PSAM 2020, Vol. 1351, 2020, p. 1346.
- [26] B. Zong, Q. Song, M.R. Min, W. Cheng, C. Lumezanu, D. Cho, H. Chen, Deep autoencoding gaussian mixture model for unsupervised anomaly detection, in: International Conference on Learning Representations, 2018.
- [27] E. Hallaji, R. Razavi-Far, M. Saif, E. Herrera-Viedma, Label noise analysis meets adversarial training: A defense against label poisoning in federated learning, *Knowl.-Based Syst.* 266 (2023) 110384.
- [28] E. Hallaji, R. Razavi-Far, M. Wang, M. Saif, B. Fardanesh, A stream learning approach for real-time identification of false data injection attacks in cyber-physical power systems, *IEEE Trans. Inf. Forensics Secur.* 17 (2022) 3934–3945.
- [29] X. Qu, L. Yang, K. Guo, L. Ma, T. Feng, S. Ren, M. Sun, Statistics-enhanced direct batch growth self-organizing mapping for efficient DoS attack detection, *IEEE Access* 7 (2019) 78434–78441.
- [30] H. Wang, Z. Cao, B. Hong, A network intrusion detection system based on convolutional neural network, *J. Intell. Fuzzy Systems* 38 (6) (2020) 7623–7637.
- [31] Z. Wu, J. Wang, L. Hu, Z. Zhang, H. Wu, A network intrusion detection method based on semantic Re-encoding and deep learning, *J. Netw. Comput. Appl.* 164 (2020) 102688.
- [32] L. Zhang, X. Lu, Z. Chen, T. Liu, Q. Chen, Z. Li, Adaptive deep learning for network intrusion detection by risk analysis, *Neurocomputing* 493 (2022) 46–58.
- [33] S. Mahdavi, A.A. Ghorbani, Application of deep learning to cybersecurity: A survey, *Neurocomputing* 347 (2019) 149–176.
- [34] M. Choraś, M. Pawlicki, Intrusion detection approach based on optimised artificial neural network, *Neurocomputing* 452 (2021) 705–715.
- [35] M. Lan, J. Luo, S. Chai, R. Chai, C. Zhang, B. Zhang, A novel industrial intrusion detection method based on threshold-optimized CNN-bilstm-attention using ROC curve, in: 2020 39th Chinese Control Conference, CCC, IEEE, 2020, pp. 7384–7389.
- [36] G. Li, Y. Shen, P. Zhao, X. Lu, J. Liu, Y. Liu, S.C. Hoi, Detecting cyberattacks in industrial control systems using online learning algorithms, *Neurocomputing* 364 (2019) 338–348.
- [37] S.K. Sahu, D.P. Mohapatra, J.K. Rout, K.S. Sahoo, Q.-V. Pham, N.-N. Dao, A LSTM-FCNN based multi-class intrusion detection using scalable framework, *Comput. Electr. Eng.* 99 (2022) 107720.
- [38] D. Singh, B. Singh, Investigating the impact of data normalization on classification performance, *Appl. Soft Comput.* 97 (2020) 105524.
- [39] M. Mayo, M. Daoud, Data normalisation using differential evolution and aggregated logistic functions, in: 2019 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2019, pp. 920–927.
- [40] Z. Nazari, D. Kang, M.R. Asharif, Y. Sung, S. Ogawa, A new hierarchical clustering algorithm, in: 2015 International Conference on Intelligent Informatics and Biomedical Sciences, ICIIBMS, IEEE, 2015, pp. 148–152.
- [41] Y. Ma, Z. Zhang, Travel mode choice prediction using deep neural networks with entity embeddings, *IEEE Access* 8 (2020) 64959–64970.

- [42] S. Singh Panwar, Y. Raiwani, L.S. Panwar, Evaluation of network intrusion detection with features selection and machine learning algorithms on CICIDS-2017 dataset, in: International Conference on Advances in Engineering Science Management & Technology (ICAESMT)-2019, Uttaranchal University, Dehradun, India, 2019.



Yaoyao Lu received the B.S. degree in Beijing Institute of Technology, Beijing, China in 2021.

He is currently studying for the M.S. degree in Beijing Institute of Technology. His current research interests include Industrial Internet of Things security and intrusion detection for Industrial Internet of Things.

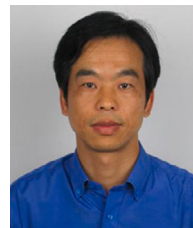


Senchun Chai received the B.S. and M.S. degrees in Beijing Institute of Technology, Beijing, China in 2001 and 2004. He received Ph.D. and post-doctor degree in advanced engineering from University of Glamorgan, UK, in 2007 and 2009.

From 2009 to 2010, he was a visiting scholar at University of Illinois, USA. Since 2010, he is an associate professor of School of Automation, Beijing Institute of Technology. His current research interests include network control, multi-agent and flight control.



Yuhan Suo received the B.S. degree from the College of Intelligent Systems Science and Engineering, Harbin Engineering University, Harbin, China, in 2021. He is currently pursuing the M.S. degree with the School of Automation, Beijing Institute of Technology, Beijing, China. His research interests include cyber-physical system security, state estimation and sensor fusion.



Fenxi Yao received the B.S. and M.S. degree in Automation from Beijing Institute of Technology, Beijing, China in 1984 and 1987.

Since 1987, he was an Assistant Professor with Beijing Institute of Technology. His current research interests include computer control system design and application, industrial control equipment design, industrial control network application, protocol analysis and Industrial Internet of Things security.



Chen Zhang received the B.S. and M.S. degrees in Beijing Institute of Technology, Beijing, China in 2018 and 2021.

His current research interests include Industrial Internet of Things, intrusion detection and industrial control system network security.